

GPU-Accelerated Machine Learning Kernels and Inference Workloads

Zarek Lu
Rice University
Houston, Texas, USA
zfl1@rice.edu

Abstract

This project studies GPU performance engineering across a collection of dense, sparse, and machine-learning workloads. I implemented and evaluated custom CUDA kernels for sparse matrix computation and two-dimensional convolution, built LeNet-5 inference using cuDNN and cuBLAS, and implemented graph convolutional network inference using cuSPARSE and cuBLAS. The experiments examine how work decomposition, shared-memory reuse, warp-level parallelism, library algorithm selection, operator fusion, and irregular sparse memory access affect application performance.

The strongest custom-kernel results include a 2.78× SpMM kernel speedup with a 43.4% reduction in end-to-end sparse-pipeline latency and tiled-convolution throughput of approximately 1.15 TFLOP/s with up to a 1.28× speedup over a naive implementation. The library-backed experiments further show strong batch-size scaling in LeNet inference and substantial dataset-dependent differences in GCN sparse-memory bandwidth. Together, these experiments illustrate how GPU performance depends not only on arithmetic throughput, but also on data movement, parallel work assignment, synchronization, and application-level orchestration.

1 Introduction

GPU acceleration is central to modern machine-learning systems, but achieving useful performance requires more than simply moving computation onto a GPU. The structure of a workload determines whether execution is limited by arithmetic throughput, memory traffic, synchronization, kernel-launch overhead, or insufficient parallelism. Dense convolution, sparse matrix multiplication, neural-network inference, and graph computation expose these constraints in different ways.

This project combines four related GPU performance-engineering studies:

- custom CUDA implementations of sparse SDDMM and SpMM using compressed sparse row (CSR) storage;
- naive and shared-memory tiled two-dimensional convolution;
- LeNet-5 inference using cuDNN and cuBLAS;
- two-layer graph convolutional network inference using cuSPARSE and cuBLAS.

The experiments span multiple levels of abstraction. The sparse and convolution studies expose thread mapping, memory access, shared memory, and warp-level parallelism directly. The LeNet and GCN studies instead delegate major operators to NVIDIA libraries and focus on descriptor setup, algorithm selection, workspace management, fusion, sparse representation, and end-to-end orchestration.

Table 1: Summary of principal GPU performance results.

Workload	Optimization	Principal result
Sparse SpMM	Warp-level parallelism	2.78× kernel speedup
Sparse pipeline	Optimized SpMM stage	43.4% lower latency
2D convolution	Shared-memory tiling	1.15 TFLOP/s, up to 1.28×
LeNet inference	cuDNN/cuBLAS + fusion	0.75 GFLOP/s best measured
GCN inference	cuSPARSE/cuBLAS + fusion	816.9k edges/s best Cora

The common goal is to understand how computation can be organized so that useful work dominates data movement and execution overhead.

2 Experimental Scope

The project was developed primarily in C++ and CUDA and used NVIDIA’s CUDA library stack where appropriate. The principal technologies were CUDA, cuDNN, cuBLAS, and cuSPARSE.

The four studies measure different scopes, so their throughput values should not be interpreted as directly comparable benchmarks. The convolution experiment measures individual custom kernels, the sparse experiment includes both kernel-level and pipeline-level measurements, and the LeNet and GCN experiments measure multi-operation inference paths.

The principal results are summarized in Table 1.

3 Sparse SDDMM and SpMM

3.1 Problem Structure

The sparse experiments use compressed sparse row storage. A sparse matrix is represented through row offsets, column indices, and nonzero values rather than as a dense matrix containing explicit zeros.

The processing pipeline includes sampled dense-dense matrix multiplication (SDDMM) followed by sparse-dense matrix multiplication (SpMM). Conceptually, SpMM computes

$$C = AX,$$

where A is sparse and X and C are dense.

Sparse computation differs from dense matrix multiplication because row lengths and memory-access patterns are irregular. A simple thread mapping can therefore leave substantial GPU parallelism unused.

3.2 Baseline Work Decomposition

The baseline SDDMM path assigns individual sparse work items to CUDA threads. The baseline SpMM implementation uses a straightforward work mapping in which a thread is responsible for substantial computation associated with an output row.

This organization is easy to reason about and provides a useful correctness baseline, but it exposes relatively little parallelism within each sparse row.

3.3 Warp-Level Optimization

The optimized SpMM implementation reorganizes row processing around warp-level parallelism. Instead of assigning the entire row computation to a single thread, lanes within a warp cooperate across sparse entries and output dimensions.

The mathematical operation remains unchanged, but the work decomposition exposes more useful parallelism and reduces serial work performed by an individual thread.

The optimized SpMM kernel achieved

2.78×

the performance of the baseline kernel.

When measured as part of the complete sparse processing path, the optimization reduced end-to-end pipeline latency by

43.4%.

The smaller application-level improvement is expected because SpMM is only one component of the complete sparse workload. Other stages are not accelerated by the same factor.

3.4 Interpretation

This experiment demonstrates the distinction between optimizing an individual GPU kernel and optimizing an entire application.

The 2.78× result describes the SpMM kernel itself, whereas the 43.4% reduction describes the larger sparse pipeline. Reporting both values is important because a large kernel-level speedup does not necessarily translate proportionally to end-to-end latency.

The result also emphasizes that sparse performance is strongly affected by work assignment. Sparse formats expose less regular parallelism than dense operations, making thread and warp mapping an important optimization variable.

4 Two-Dimensional Convolution

4.1 Implementations

I implemented both naive and tiled CUDA convolution kernels.

The naive implementation assigns one thread to each output element. Each thread directly loads the input and filter values required to compute that output from global memory. Although this implementation is straightforward and easy to verify, it does not explicitly exploit the substantial reuse inherent in convolution.

The tiled implementation uses 16×16 thread blocks. With stride 1 and a 3×3 kernel, each block computes a 16×16 output tile while cooperatively staging an 18×18 region of the input into shared memory.

For each input-output channel pair, the corresponding 3×3 filter is also staged for reuse within the block.

Table 2: Summary of measured convolution throughput.

Implementation	Throughput	Max speedup
Naive	844–899 GFLOP/s	—
Tiled	931–1153 GFLOP/s	1.28×

This organization exploits two primary forms of reuse:

- neighboring output pixels reuse overlapping regions of the input;
- all threads in the block reuse the same filter values for a given channel pair.

4.2 Benchmark Configuration

The benchmark swept spatial dimensions

256×256 , 512×512 , 1024×1024

and output-channel counts

32, 64, 128.

Input channels remained fixed at 32, with a 3×3 convolution kernel, stride 1, and padding 1.

4.3 Measured Performance

Across the benchmark sweep, the naive implementation achieved approximately

844–899 GFLOP/s,

while the tiled implementation achieved approximately

931–1153 GFLOP/s.

The tiled implementation therefore reached approximately

1.15 TFLOP/s

in the strongest measured configuration.

The measured speedup ranged from approximately 1.07× for the smallest case to

1.28×

for the largest case, with an average improvement of approximately

1.18×

4.4 Memory-Reuse Analysis

For a single input channel, the tiled implementation stages an 18×18 input region and a 3×3 filter:

$18 \times 18 + 3 \times 3 = 333$

single-precision values.

At four bytes per value, this corresponds to approximately

1332 bytes

of shared memory for the staged input and filter data.

The improvement is meaningful but not dramatic. Hardware caches already recover some locality for the naive implementation, and the tiled design still reloads input regions for different output channels and filter weights across spatial blocks. Each thread also computes only one output element, limiting register-level reuse.

The experiment therefore demonstrates both the benefit and the limitations of straightforward shared-memory tiling.

5 LeNet-5 Inference

5.1 Architecture

The next experiment implements LeNet-5 inference using NVIDIA’s optimized GPU libraries rather than hand-written convolution kernels.

The forward path follows the standard LeNet structure:

$\text{Conv}(6, 5 \times 5) \rightarrow \text{Pool} \rightarrow \text{Conv}(16, 5 \times 5) \rightarrow \text{Pool} \rightarrow \text{FC}_{120} \rightarrow \text{FC}_{84} \rightarrow \text{FC}_{10}$

cuDNN performs convolution, activation, and pooling, while cuBLAS SGEMM performs the fully connected layers.

The implementation required explicit management of tensor descriptors, filter descriptors, convolution descriptors, activation and pooling descriptors, workspace allocation, device memory, and the transition between convolutional and dense representations.

5.2 Baseline and Fused Paths

The baseline convolution blocks perform separate operations for convolution, bias addition, activation, and pooling.

The fused implementation retains cuDNN convolution and pooling but replaces the separate convolution bias-addition and activation calls with a custom CUDA kernel that applies the bias and tanh activation in a single pass over the convolution output.

The goal was to evaluate whether reducing intermediate operations could improve end-to-end inference while retaining cuDNN for the main convolution computation.

5.3 Benchmark Sweep

The experiment varied:

- batch sizes 32, 64, and 128;
- cuDNN algorithms `implicit_gemm`, `implicit_precomp`, and `fft`;
- baseline and fused execution paths.

The measured metrics were end-to-end latency, effective GFLOP/s, and cuDNN convolution workspace size.

5.4 Batch-Size Scaling

The clearest overall trend is the effect of batch size.

Batch-32 configurations remained within approximately

0.08–0.21 GFLOP/s,

while batch-128 configurations reached approximately

0.63–0.75 GFLOP/s.

The best measured configuration was fused `implicit_gemm` at batch 128:

Table 3: Measured LeNet-5 inference results.

Mode	Algorithm	Batch	Time (ms)	GFLOP/s
baseline	<code>implicit_gemm</code>	32	127.90	0.21
fused	<code>implicit_gemm</code>	32	129.35	0.21
baseline	<code>implicit_precomp</code>	32	241.25	0.11
fused	<code>implicit_precomp</code>	32	151.75	0.18
baseline	<code>fft</code>	32	317.92	0.08
fused	<code>fft</code>	32	130.59	0.20
baseline	<code>implicit_gemm</code>	64	222.64	0.24
fused	<code>implicit_gemm</code>	64	166.58	0.32
baseline	<code>implicit_precomp</code>	64	139.55	0.38
fused	<code>implicit_precomp</code>	64	151.53	0.35
baseline	<code>fft</code>	64	159.75	0.33
fused	<code>fft</code>	64	146.06	0.37
baseline	<code>implicit_gemm</code>	128	153.36	0.70
fused	<code>implicit_gemm</code>	128	141.89	0.75
baseline	<code>implicit_precomp</code>	128	168.38	0.63
fused	<code>implicit_precomp</code>	128	143.68	0.74
baseline	<code>fft</code>	128	150.26	0.71
fused	<code>fft</code>	128	144.92	0.74

0.75 GFLOP/s

with an end-to-end latency of

141.89 ms.

This scaling indicates that larger batches allow fixed launch and library-call overhead to be amortized over more useful computation.

5.5 Algorithm and Workspace Tradeoffs

Workspace requirements differed substantially across the tested cuDNN algorithms.

`implicit_gemm` required no additional workspace in the recorded runs.

`implicit_precomp` required only small temporary allocations: 5760 bytes at batch 32 and 1664 bytes at batches 64 and 128.

The FFT algorithm required much larger workspace allocations, increasing from approximately 1.67 MB at batch 32 to 6.69 MB at batch 128.

At batch 128, however, all three algorithms produced relatively similar end-to-end throughput. This makes `implicit_gemm` particularly attractive in the measured environment because it achieved the highest observed throughput without additional convolution workspace.

The result illustrates that GPU library algorithm selection should consider application-level latency and temporary-memory requirements together rather than treating either metric in isolation.

6 Graph Convolutional Network Inference

6.1 Architecture

The final experiment implements a two-layer graph convolutional network using CUDA libraries rather than a framework-level graph abstraction.

Table 4: Measured GCN inference results.

Graph	Impl.	Hidden	Time	Edges/s	GB/s	GPU Acc.
Cora	baseline	64	29.30	452654.71	3.27	0.100812
Cora	fused	64	29.25	453398.89	3.28	0.100812
Cora	baseline	128	16.45	806343.37	6.08	0.132201
Cora	fused	128	16.77	790819.25	5.96	0.132201
Cora	baseline	256	16.95	782636.62	6.38	0.100812
Cora	fused	256	16.24	816861.41	6.66	0.100812
Citeseer	baseline	64	16.80	737102.14	14.09	0.084541
Citeseer	fused	64	16.75	739142.04	14.13	0.084541
Citeseer	baseline	128	16.74	739640.76	14.38	0.161836
Citeseer	fused	128	16.18	765599.82	14.88	0.161836
Citeseer	baseline	256	16.88	733501.88	14.74	0.086353
Citeseer	fused	256	16.37	756697.35	15.20	0.086353

The normalized adjacency matrix is

$$\hat{A} = D^{-1/2}(A + I)D^{-1/2},$$

and the forward computation follows

$$\hat{A}XW_1 \rightarrow \text{ReLU} \rightarrow \hat{A}HW_2.$$

cuSPARSE SpMM performs sparse graph aggregation, while cuBLAS SGEMM performs the learned dense transformations.

The benchmark evaluates Cora and Citeseer using hidden dimensions 64, 128, and 256 for both baseline and fused execution paths.

6.2 Graph Preprocessing

The graph is stored in CSR format. For each dataset, the implementation loads row offsets, column indices, node features, and labels.

Self-loops are added and normalized adjacency coefficients are materialized before the forward pass. Consequently, the reported runtime measurements focus on inference rather than repeatedly including graph preprocessing.

6.3 Baseline and Fused Paths

The baseline path keeps the major forward-pass operations separate. Each hidden layer performs sparse aggregation using cuSPARSE SpMM, dense weight multiplication using cuBLAS SGEMM, and activation processing.

The fused path preserves the same sparse and dense library operations but combines activation and dropout-style hidden-layer postprocessing into a single kernel.

Dropout was effectively disabled during the recorded inference measurements, so the main experimental difference is reduced hidden-layer postprocessing overhead rather than a change to the model computation.

6.4 Performance Results

Table 4 reports the complete twelve-configuration sweep.

6.5 Fusion Behavior

Fusion produced modest and workload-dependent improvements rather than a universal speedup.

The clearest Cora improvement occurs at hidden size 256:

$$16.95 \text{ ms} \rightarrow 16.24 \text{ ms},$$

corresponding to an approximately

$$4.2\%$$

reduction in latency.

Throughput simultaneously increased from approximately 782.6 thousand edges/s to

$$816.9 \text{ k edges/s}.$$

For Citeseer at hidden size 128, fusion reduced latency from

$$16.74 \text{ ms}$$

to

$$16.18 \text{ ms},$$

an improvement of approximately

$$3.3\%.$$

At hidden size 256, Citeseer latency decreased from 16.88 ms to 16.37 ms.

Fusion was not beneficial in every configuration. Cora at hidden size 128 regressed slightly from 16.45 ms to 16.77 ms.

This demonstrates that reducing operator count does not automatically imply lower wall-clock time. The effect depends on workload structure and on the balance between sparse aggregation, dense computation, and fixed runtime overhead.

6.6 Dataset-Dependent Sparse Bandwidth

One of the strongest findings is the difference in effective SpMM bandwidth between Cora and Citeseer.

Cora reaches a maximum of approximately

$$6.66 \text{ GB/s},$$

while Citeseer reaches

$$15.20 \text{ GB/s}.$$

The difference is greater than a factor of two despite both workloads being citation-style graph datasets of broadly similar scale.

This indicates that sparse performance cannot be predicted from node or edge count alone. CSR row distribution, graph degree structure, access locality, and irregular memory traversal can strongly affect achieved bandwidth.

6.7 Reference Accuracy

The GPU and recorded reference accuracies match exactly to the printed precision for nearly every configuration.

The only visible discrepancy occurs on Cora at hidden size 256:

$$\text{GPU accuracy} = 0.100812,$$

while

$$\text{reference accuracy} = 0.101182.$$

The absolute difference is

$$0.00037.$$

Because this experiment evaluates inference implementation rather than model training quality, these accuracy values are primarily useful as functional consistency checks.

6.8 DGL Comparison Limitation

The intended evaluation included a direct throughput comparison against DGL.

A reliable DGL timing measurement was not obtained in the final experimental environment. The required environment could not be completed within the available cluster disk quota, and a subsequent local attempt encountered DGL compatibility issues on Windows.

Accordingly, this project does not make a measured throughput claim relative to DGL.

The available reference results support functional comparison, but they do not support a defensible statement that the cuSPARSE implementation is faster or slower than DGL.

7 Cross-Workload Analysis

7.1 Memory Locality

The convolution experiment most directly demonstrates the effect of explicit memory reuse. Shared-memory tiling increased throughput from the high-800 GFLOP/s range to more than 1 TFLOP/s by staging overlapping receptive fields for reuse within a thread block.

Sparse workloads expose a different memory problem. CSR traversal creates irregular accesses that cannot be transformed into the same regular tiled pattern.

The more than twofold difference in effective GCN SpMM bandwidth between Cora and Citeseer demonstrates how strongly graph structure can affect sparse memory behavior.

7.2 Parallel Work Decomposition

The optimized sparse SpMM experiment shows that the granularity of work assignment can be as important as the arithmetic being performed.

Moving from a simple thread-oriented work mapping to warp-level cooperation produced a 2.78 $\times$ kernel speedup without changing the underlying mathematical operation.

This illustrates a broader GPU-programming principle: an algorithm may expose substantial theoretical parallelism while an inefficient work mapping fails to make that parallelism available to the hardware.

2026-09-08 07:30. Page 5 of 1-6.

7.3 Custom Kernels and Vendor Libraries

The project spans two complementary styles of GPU development.

Custom CUDA kernels provide direct control over:

- thread and warp assignment;
- shared-memory staging;
- synchronization;
- memory-access behavior;
- arithmetic and data reuse.

Vendor-library implementations move much of the low-level optimization into cuDNN, cuBLAS, and cuSPARSE, but introduce different systems challenges:

- descriptor construction;
- data layout;
- workspace allocation;
- library algorithm selection;
- operator sequencing;
- fusion boundaries.

The LeNet and GCN studies show that substantial performance reasoning remains necessary even when core mathematical operators are delegated to optimized libraries.

7.4 Fusion Is Workload-Dependent

Both LeNet and GCN include modest fusion experiments.

In LeNet, the fused path is frequently faster and remains slightly ahead for all three tested convolution algorithms at batch 128.

In GCN inference, fusion improves several configurations but slightly hurts Cora at hidden size 128.

The experiments therefore support a more careful conclusion than simply stating that fusion improves performance. Fusion removes some intermediate work, but its practical benefit depends on whether that work is significant relative to the remaining computation and runtime overhead.

7.5 Kernel-Level and End-to-End Performance

The sparse study provides the clearest example of the distinction between local and global optimization.

The SpMM kernel improves by 2.78 $\times$, while the complete sparse pipeline improves by 43.4%.

Likewise, LeNet end-to-end throughput depends on convolution, pooling, activation, dense layers, memory movement, and library-call overhead. Improving one operator does not necessarily produce a proportional application-level improvement.

Performance results should therefore always identify the scope of the measurement.

8 Correctness and Validation

Performance optimization is only meaningful if the optimized implementation remains numerically or functionally correct.

Across the project, optimized code was evaluated against available reference implementations before performance results were interpreted.

For the custom sparse and convolution workloads, correctness checks were used to verify that changes to thread mappings and memory layouts preserved the intended computation.

The LeNet implementation included reference verification of inference outputs.

The GCN evaluation compared GPU behavior against a recorded reference path and produced closely matching classification results throughout the twelve-configuration sweep.

The general workflow used throughout the project was therefore

Implement → Validate → Measure → Optimize → Revalidate

This separation is particularly important for CUDA optimization because indexing, synchronization, and memory-layout errors can produce fast but incorrect kernels.

9 Limitations

The reported results should be interpreted as measurements of these implementations in their experimental environment rather than as universal performance characteristics of the corresponding algorithms.

The four studies also measure different scopes. Custom convolution throughput represents an individual operator, while LeNet and GCN timings include complete inference paths containing multiple library calls and intermediate stages.

The sparse and tiled-convolution implementations explore specific optimization strategies rather than exhaustive kernel searches.

The GCN study does not contain a valid DGL timing comparison, so no framework-level performance claim is made.

Several optimization directions remain open:

- processing multiple convolution outputs per thread to improve register reuse;
- reusing convolution tiles more aggressively across output channels;
- specializing sparse kernels for different row-length distributions;
- using deeper profiler measurements to isolate memory stalls and synchronization costs;
- evaluating more aggressive operator fusion in LeNet;
- testing GCN behavior on larger and structurally different graphs.

10 Lessons Learned

The project reinforced several general lessons about GPU performance engineering.

First, data movement frequently matters as much as arithmetic. The convolution experiment improved performance primarily by reducing redundant global-memory traffic, while the GCN experiment showed that sparse memory bandwidth can differ substantially across graph structures.

Second, parallelism must be exposed through an effective work decomposition. Warp-level restructuring substantially improved sparse SpMM even though the underlying mathematical operation did not change.

Third, optimized libraries do not eliminate systems-level performance decisions. Descriptor management, workspace size, data layout, algorithm choice, and fusion can materially affect complete inference pipelines.

Fourth, performance results need context. Kernel throughput, end-to-end latency, memory consumption, and correctness measure different properties and should not be collapsed into a single number.

Finally, optimization should remain empirical. Shared-memory tiling, batching, and fusion all produced measurable improvements in some configurations, but their value depended on workload size and structure.

11 Conclusion

This project explored GPU machine-learning computation across multiple levels of abstraction, from custom CUDA kernels to complete inference paths built from NVIDIA's optimized libraries.

The strongest measured custom-kernel results were a 2.78× sparse SpMM kernel speedup, a 43.4% reduction in end-to-end sparse-pipeline latency, and tiled convolution reaching approximately 1.15 TFLOP/s with up to a 1.28× improvement over the naive CUDA implementation.

The LeNet experiment demonstrated substantial throughput scaling with batch size, reaching 0.75 GFLOP/s in the best measured batch-128 configuration while exposing meaningful workspace and algorithm selection tradeoffs.

The GCN experiment produced a working cuSPARSE/cuBLAS inference pipeline across Cora and Citeseer. The best Cora configuration reached 16.24 ms and approximately 816.9 thousand edges/s, while Citeseer reached 15.20 GB/s effective SpMM bandwidth. The large bandwidth difference between datasets reinforces the importance of memory locality and graph structure in sparse GPU workloads.

Taken together, the experiments demonstrate that GPU performance depends on the interaction between algorithm structure, work decomposition, memory access, optimized library behavior, and end-to-end system organization. Effective optimization therefore requires both low-level hardware reasoning and careful application-level measurement.